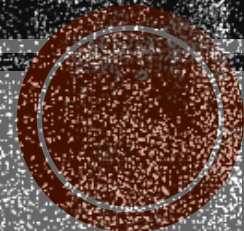




BIG CITY PUBLIC TRANSPORT SIMULATION  
DOKUZ EYLÜL UNIVERSITY - FACULTY OF  
ENGINEERING ELECTRICAL AND  
ELECTRONICS ENGINEERING DEPARTMENT  
COURSE: EED 1010 ALGORITHMS AND  
PROGRAMMING

- Prepared by:  
Omer Ege Nisli (2024502054)
- Yigithan Yildirim (2024502076)
- Asai Ghazi Mir Saeid (2022502017)
- Professor:  
Dr. Özlem Öztürk







```
Bus [D2] Passengers:  
55: D-P (L:-)  
78: K-I (L:80,81)  
95: N-L (L:100,101)  
110: M-I (L:115)  
Bus [D2] Luggage:  
| 80 | 81 | 100 | 101 | 115 |  
-----
```

- This project simulates a smart public transportation system operating in a city. It includes six bus lines, twelve buses, and sixteen distinct stops from A to P.
- The buses follow fixed routes and interact with dynamically generated passengers. Each passenger has a randomly assigned start and end stop, and may carry up to two pieces of luggage.
- Passengers board buses following a FIFO (First-In-First-Out) system, with the condition that the bus must have enough capacity for both the person and their luggage.
- The simulation is written entirely in the C programming language.
- It demonstrates various concepts such as structured programming, memory management, modular design, and real-time console output.
- Buses move step by step across a standard-rendered city grid, and statistics like passenger count, bus fullness, and queue sizes are continuously displayed to the user in real







- The system consists of three core components:

1. Data Layer: Includes the data structures defining passengers, buses, stops, the city map, and bus lines.

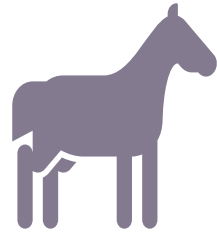
2. Game Engine: Responsible for moving buses, boarding and unloading passengers, checking luggage capacity, and simulating time.

3. Analytics and Visualization Layer.\* Draws the city grid, shows bus locations, and prints live statistics. These layers are modular, allowing for maintainability and future expansion





Each passenger has a unique ID, a start and end stop, and a luggage count between 0 and 2.



All passengers queue at their start stop. Stops use a circular queue system.



Buses are defined by their line name and ID. Each can carry up to 8 passengers or 8 pieces of luggage.



Buses move on fixed routes and reverse direction at endpoints.



Boarding logic checks for both passenger and luggage space.









# ERROR

- Challenges and Solutions:
- Luggage Overload: Handled by preventing boarding when capacity is full
- Queue Corruption: Fixed with index boundary checks
- Screen Flicker: Reduced with controlled refresh rates
- Passenger Unloading Errors: Solved by manually shifting array contents during deletions







# WHAT'S NEXT?

- Conclusion and Future Work:
  - This project helped us apply structured C programming in a real-time simulation context.
  - We gained experience in data management, simulation logic, and user interfaces.
  - Possible Future Enhancements:
  - Colorful Graphical interface
  - Multi-threading for smoother performance
  - Passenger impatience and timing logic
- 